

UNIT 2

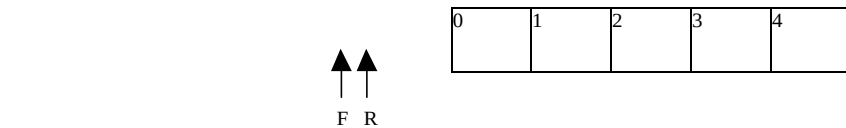
QUEUE

Basic Queue Operations:

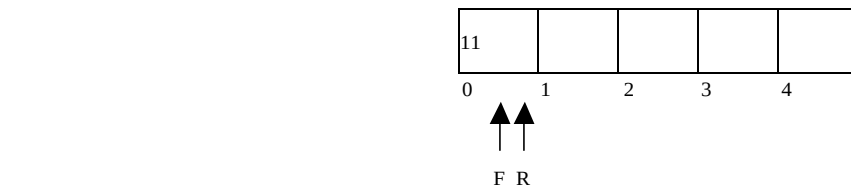
A queue is a data structure that is best described as "first in, first out". A queue is another special kind of list, where items are inserted at one end called the rear and deleted at the other end called the front. A real world example of a queue is people waiting in line at the bank. As each person enters the bank, he or she is "enqueued" at the back of the line. When a teller becomes available, they are "dequeued" at the front of the line.

Representation of a Queue using Array:

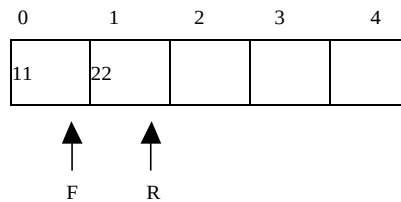
Let us consider a queue, which can hold maximum of five elements. Initially the queue is empty.



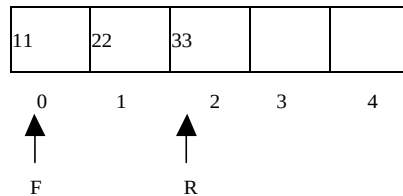
Now, insert 11 to the queue. Then queue status will be:



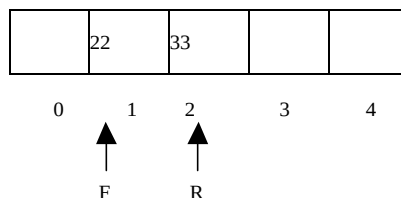
Next, insert 22 to the queue. Then the queue status is:



Again insert another element 33 to the queue. The status of the queue is:

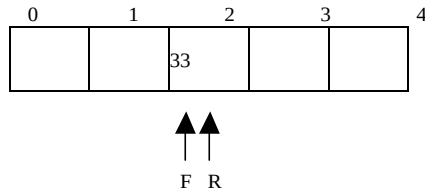


Now, delete an element. The element deleted is the element at the front of the queue. So the status of the queue is:



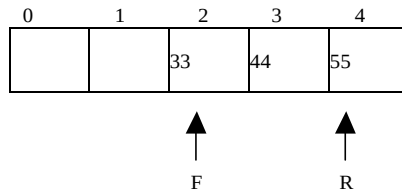
Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted. The queue

status is as follows:



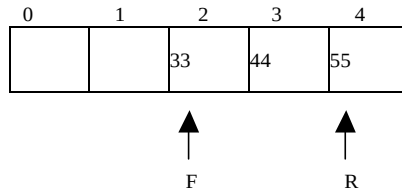
REAR = 2
FRONT = FRONT + 1 = 2

Now, insert new elements 44 and 55 into the queue. The queue status is:



REAR = 4
FRONT = 2

Next insert another element, say 66 to the queue. We cannot insert 66 to the queue as the rear crossed the maximum size of the queue (i.e., 5). There will be queue full signal. The queue status is as follows:



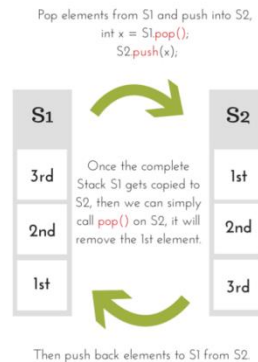
REAR = 4 FRONT = 2

Now it is not possible to insert an element 66 even though there are two vacant positions in the linear queue. To overcome this problem the elements of the queue we can use **circular queue**.

Implement Queue using Stacks

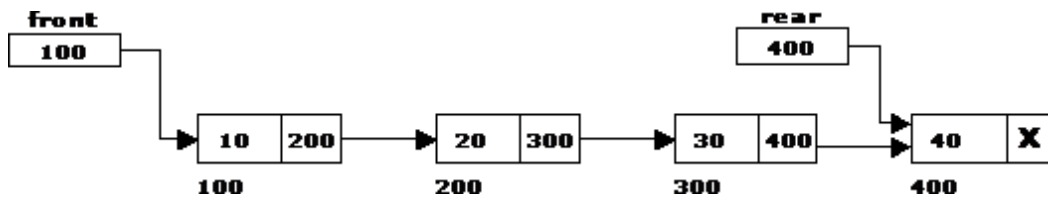
While implementing a queue data structure using stacks, we will have to consider the natural behavior of stack too, which is First in Last Out.

For performing enqueue we require only one stack as we can directly push data onto the stack, but to perform dequeue we will require two Stacks, because we need to follow queue's FIFO property and if we directly pop any data element out of Stack, it will follow LIFO approach (Last in First Out).



Linked List Implementation of Queue:

We can represent a queue as a linked list. In a queue data is deleted from the front end and inserted at the rear end. We can perform similar operations on the two ends of a list. We use two pointers front and rear for our linked queue implementation.



Applications of Queues:

1. It is used to schedule the jobs to be processed by the CPU.
2. When multiple users send print jobs to a printer, each printing job is kept in the printing queue. Then the printer prints those jobs according to first in first out (FIFO) basis.
3. Breadth first search uses a queue data structure to find an element from a graph.

Disadvantages of Linear Queue:

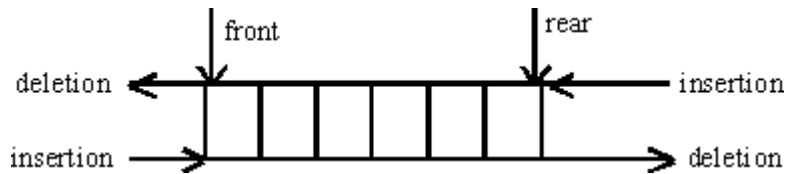
There are two problems associated with linear queue. They are:

- ☐ Time consuming: linear time to be spent in shifting the elements to the beginning of the queue.
- ☐ Signaling queue full: even if the queue is having vacant position.

DEQUE(Double Ended Queue):

A **double-ended queue (deque)**, often abbreviated to **deque**, pronounced *deck*) generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail). It is also often called a **head-tail linked list**. Like an ordinary queue, a double-ended queue is a data structure it supports the following operations: enq_front, enq_back, deq_front, deq_back, and empty. Dequeue can behave like a queue by using only enq_front and deq_front, and behaves like a stack by using only enq_front and deq_rear.

The DeQueue is represented as follows.



DEQUE can be represented in two ways they are

- 1) Input restricted DEQUE(IRD)
- 2) output restricted DEQUE(ORD)

The **output restricted** DEQUE allows deletions from only one end and **input restricted** DEQUE allow insertions at only one end. The DEQUE can be constructed in two ways they are

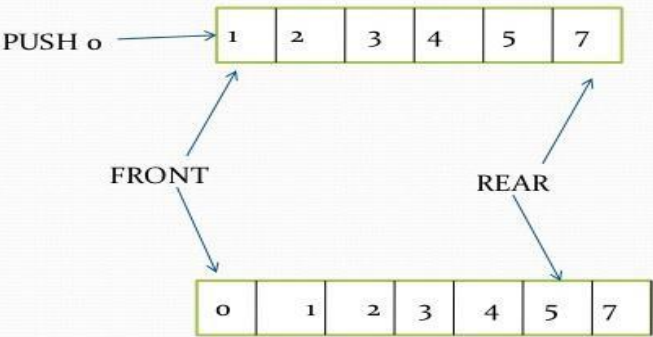
- 1) Using array
- 2) Using Linked list

Operations in DEQUE

1. Insert element at back
2. Insert element at front
3. Remove element at front

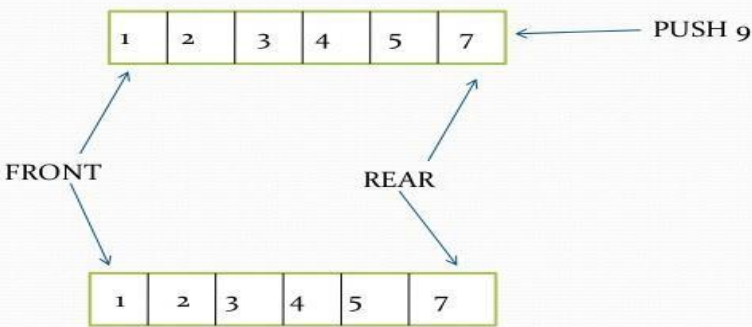
Insert_front

- insert_front() is a operation used to push an element into the front of the *Deque*.



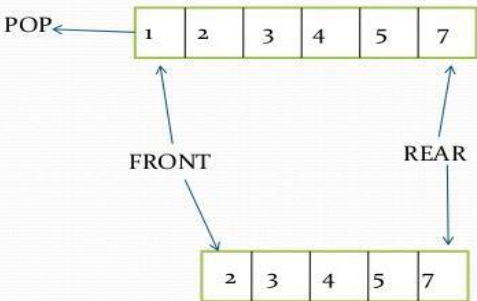
Insert_back

- insert_back() is a operation used to push an element at the back of a *Deque*.



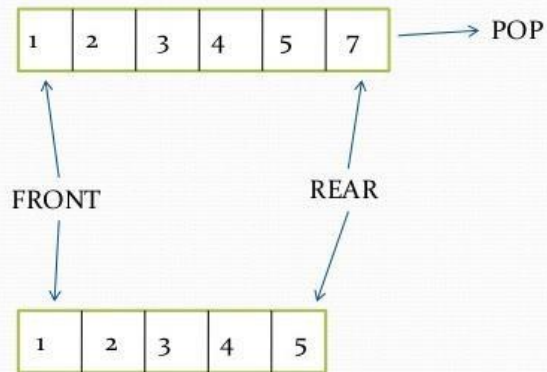
Remove_front

- remove_front() is a operation used to pop an element on front of the *Deque*.



Remove_back

• `remove_front()` is a operation used to pop an element on front of the *Deque*.



Applications of DEQUE:

1. The A-Steal algorithm implements task scheduling for several processors (multiprocessor scheduling).
2. The processor gets the first element from the deque.
3. When one of the processor completes execution of its own threads it can steal a thread from another processor.
4. It gets the last element from the deque of another processor and executes it.

Circular Queue:

Circular queue is a linear data structure. It follows FIFO principle. In circular queue pointer at the last node is connected back to the first node to make a circle.

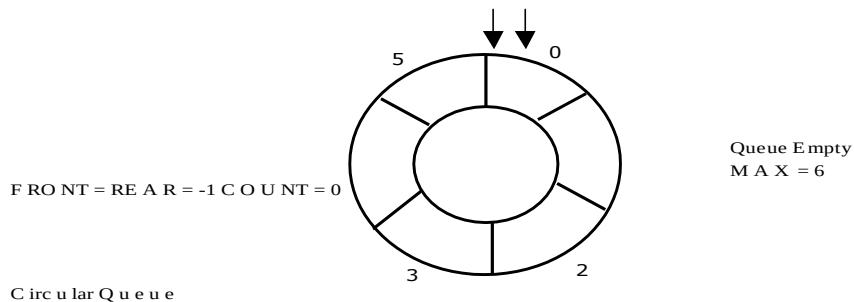
- Circular linked list follow the First In First Out principle
- Elements are added at the rear end and the elements are deleted at front end of the queue
- Both the front and the rear pointers points to the beginning of the array.
- It is also called as “**Ring buffer**”.
- Items can inserted and deleted from a queue in $O(1)$ time.

Circular Queue can be created in three ways they are:

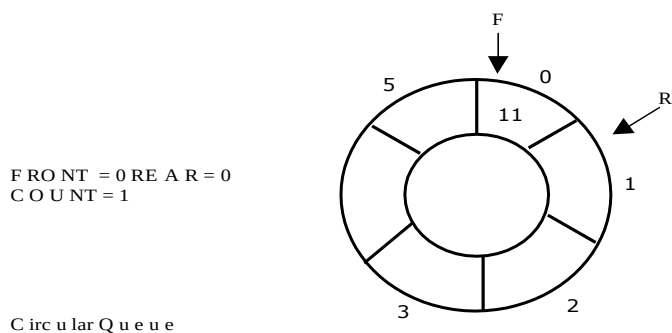
1. Using single linked list
2. Using double linked list
3. Using arrays

Representation of Circular Queue:

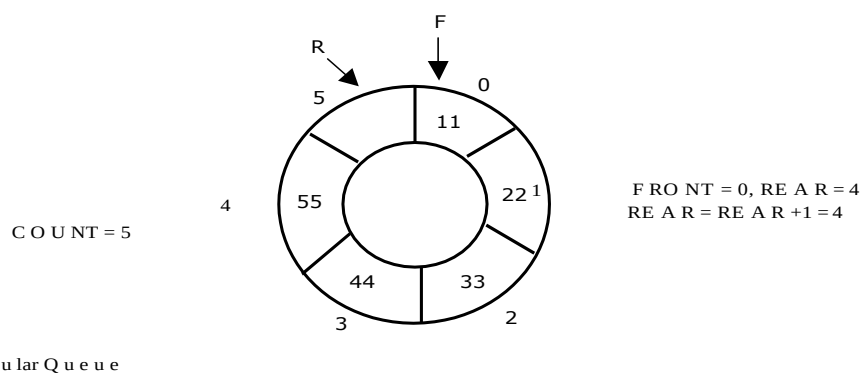
Let us consider a circular queue, which can hold maximum (MAX) of six elements. Initially the queue is empty.



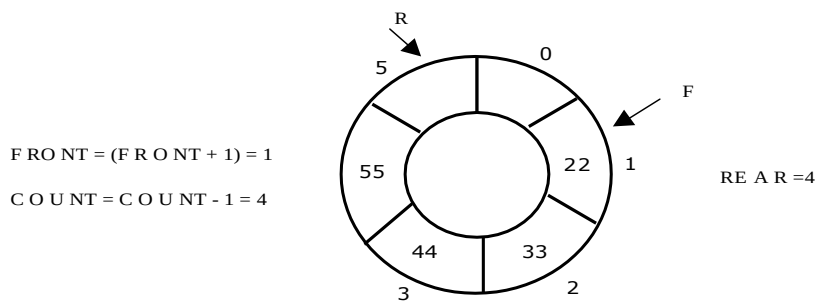
Now, insert 11 to the circular queue. Then circular queue status will be:



Insert new elements 22, 33, 44 and 55 into the circular queue. The circular queue status is:

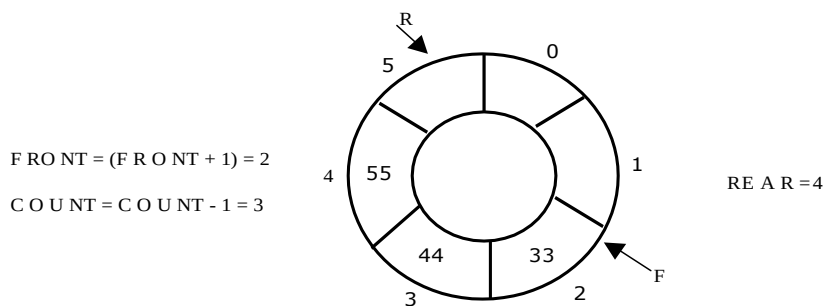


Now, delete an element. The element deleted is the element at the front of the circular queue. So, 11 is deleted. The circular queue status is as follows:



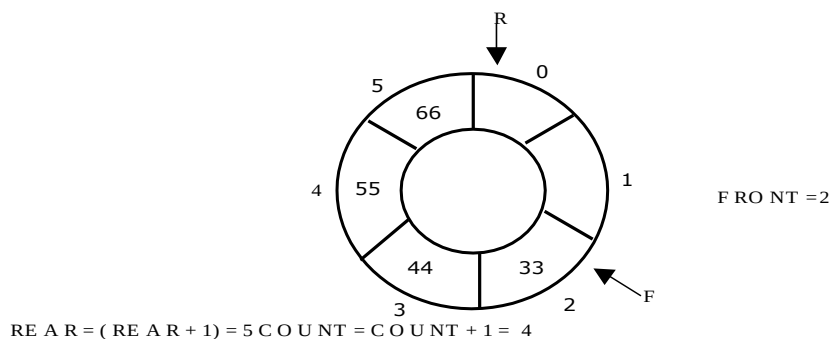
Circular Queue

Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted. The circular queue status is as follows:



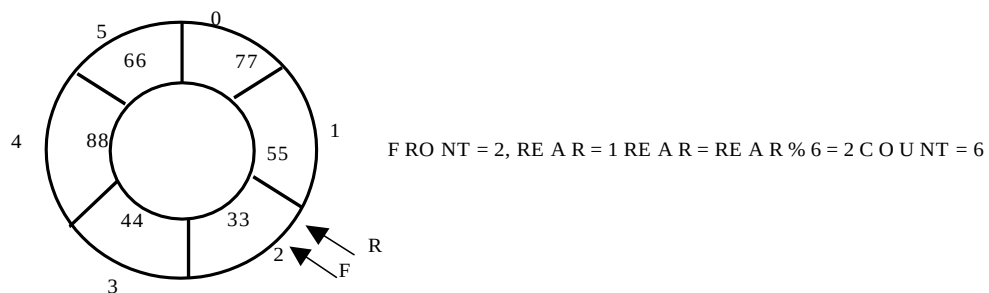
Circular Queue

Again, insert another element 66 to the circular queue. The status of the circular queue is:



Circular Queue

Now, insert new elements 77 and 88 into the circular queue. The circular queue status is:



Now, if we insert an element to the circular queue, as $COUNT = MAX$ we cannot add the element to circular queue. So, the circular queue is *full*.

Priority Queue is an abstract data type that is similar to a queue, and every element has some priority value associated with it. The priority of the elements in a priority queue determines the order in which elements are served (i.e., the order in which they are removed). If in any case the elements have same priority, they are served as per their ordering in the queue.

Properties of Priority Queue

So, a priority Queue is an extension of the [queue](#) with the following properties.

- Every item has a priority associated with it.
- An element with high priority is dequeued before an element with low priority.
- If two elements have the same priority, they are served according to their order in the queue.

Priority Queue		
Initial Queue = { }		
Operation	Return value	Queue Content
insert (C)		C
insert (O)		C O
insert (D)		C O D
remove max	O	C D
insert (I)		C D I
insert (N)		C D I N
remove max	N	C D I
insert (G)		C D I G

How to Implement Priority Queue?

Priority queue can be implemented using the following data structures:

- Arrays
- Linked list
- Heap data structure
- Binary search tree

A typical priority queue supports following operations. **insert(item, priority)**: Inserts an item with given priority. **getHighestPriority()**: Returns the highest priority item. **deleteHighestPriority()**: Removes the highest priority item.

Implementation of priority queue

Using Array: A simple implementation is to use array of following structure.

insert() operation can be implemented by adding an item at end of array in $O(1)$ time.

getHighestPriority() operation can be implemented by linearly searching the highest priority item in array. This operation takes $O(n)$ time.

deleteHighestPriority() operation can be implemented by first linearly searching an item, then removing the item by moving all subsequent items one position back.

We can also use Linked List, time complexity of all operations with linked list remains same as array. The advantage with linked list is **deleteHighestPriority()** can be more efficient as we don't have to move items.

Applications of Priority Queue:

- 1) CPU Scheduling
- 2) Graph algorithms like Dijkstra's shortest path algorithm, Prim's Minimum Spanning Tree, etc
- 3) All queue applications where priority is involved.

Round Robin Algorithm

- **Round Robin** is a CPU scheduling algorithm where each process is assigned a fixed time slot in a cyclic way.
- Round robin is a pre-emptive algorithm.
- The CPU is shifted to the next process after fixed interval time, which is called time quantum/time slice.
- The process that is preempted is added to the end of the queue.
- Round robin is one of the oldest, fairest, and easiest algorithm.

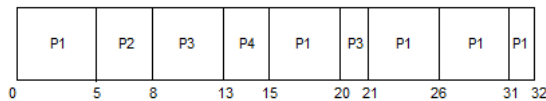
Important terms

- **Completion Time** It is the time at which any process completes its execution.
- **Turn Around Time** This mainly indicates the time Difference between completion time and arrival time. The Formula to calculate the same is: **Turn Around Time = Completion Time – Arrival Time**
- **Waiting Time(W.T):** It Indicates the time Difference between turn around time and burst time. And is calculated as **Waiting Time = Turn Around Time – Burst Time**

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2



The GANTT chart for round robin scheduling will be,



The average waiting time will be, 11 ms.

In the above diagram, arrival time is not mentioned so it is taken as 0 for all processes.

Note: If arrival time is not given for any problem statement then it is taken as 0 for all processes; if it is given then the problem can be solved accordingly.

Processes	Burst Time	Turn Around Time Turn Around Time = Completion Time – Arrival Time	Waiting Time Waiting Time = Turn Around Time – Burst Time
P1	21	32-0=32	32-21=11
P2	3	8-0=8	8-3=5
P3	6	21-0=21	21-6=15

Processes	Burst Time	Turn Around Time Turn Around Time = Completion Time – Arrival Time	Waiting Time Waiting Time = Turn Around Time – Burst Time
P4	2	15-0=15	15-2=13

LINKED LISTS

Linked lists and arrays are similar since they both store collections of data. Array is the most common data structure used to store collections of elements. Arrays are convenient to declare and provide the easy syntax to access any element by its index number. Once the array is set up, access to any element is convenient and fast.

The disadvantages of arrays are:

- The size of the array is fixed. Most often this size is specified at compile time. This makes the programmers to allocate arrays, which seems "large enough" than required.
- Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.
- Deleting an element from an array is not possible. Linked lists have their own strengths and weaknesses, but they happen to be strong where arrays are weak.
- Generally array's allocates the memory for all its elements in one block whereas linked lists use an entirely different strategy. Linked lists allocate memory for each element separately and only when necessary.

Linked List Concepts:

A linked list is a non-sequential collection of data items. It is a dynamic data structure. For every data item in a linked list, there is an associated pointer that would give the memory location of the next data item in the linked list. The data items in the linked list are not in consecutive memory locations. They may be anywhere, but the accessing of these data items is easier as each data item contains the address of the next data item.

Advantages of linked lists:

Linked lists have many advantages. Some of the very important advantages are:

1. Linked lists are dynamic data structures. i.e., they can grow or shrink during the execution of a program.
2. Linked lists have efficient memory utilization. Here, memory is not preallocated. Memory is allocated whenever it is required and it is de-allocated (removed) when it is no longer needed.
3. Insertion and Deletions are easier and efficient. Linked lists provide flexibility in inserting a data item at a specified position and deletion of the data item from the given position.
4. Many complex applications can be easily carried out with linked lists.

Disadvantages of linked lists:

1. It consumes more space because every node requires a additional pointer to store address of the next node.
2. Searching a particular element in list is difficult and also time consuming.

Types of Linked Lists:

Basically we can put linked lists into the following four items:

1. Single Linked List.
2. Double Linked List.
3. Circular Linked List.
4. Circular Double Linked List.

A single linked list is one in which all nodes are linked together in some sequential manner. Hence, it is also called as linear linked list.

A double linked list is one in which all nodes are linked together by multiple links which helps in accessing both the successor node (next node) and predecessor node (previous node) from any arbitrary node within the list. Therefore each node in a double linked list has two link fields (pointers) to point to the left node (previous) and the right node (next). This helps to traverse in

forward direction and backward direction.

A **circular linked list** is one, which has no beginning and no end. A single linked list can be made a circular linked list by simply storing address of the very first node in the link field of the last node.

A **circular double linked list** is one, which has both the successor pointer and predecessor pointer in the circular manner.

Comparison between array and linked list:

ARRAY	LINKED LIST
Size of an array is fixed	Size of a list is not fixed
Memory is allocated from stack	Memory is allocated from heap
It is necessary to specify the number of elements during declaration (i.e., during compile time).	It is not necessary to specify the number of elements during declaration (i.e., memory is allocated during run time).
It occupies less memory than a linked list for the same number of elements.	It occupies more memory.
Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.	Inserting a new element at any position can be carried out easily.
Deleting an element from an array is not possible.	Deleting an element is possible.

Trade offs between linked lists and arrays:

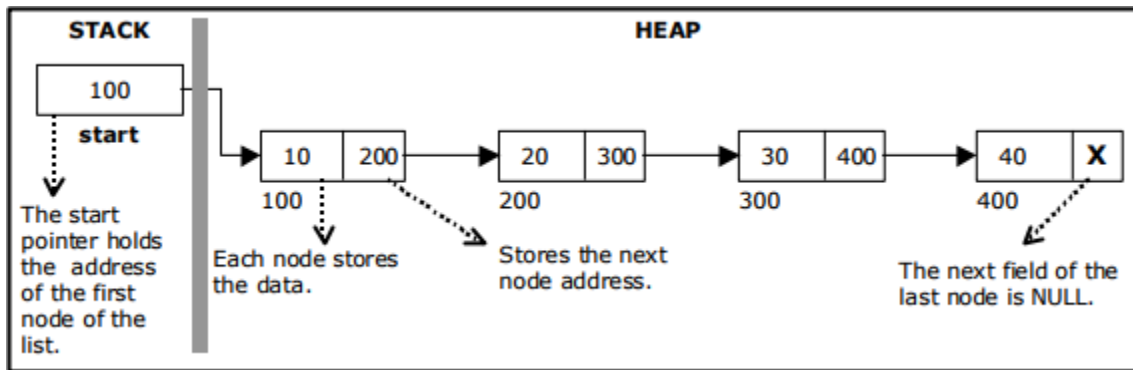
FEATURE	ARRAYS	LINKED LISTS
Sequential access	efficient	efficient
Random access	efficient	inefficient
Resigning	inefficient	efficient
Element rearranging	inefficient	efficient
Overhead per elements	none	1 or 2 links

Applications of linked list:

1. Linked lists are used to represent and manipulate polynomial. Polynomials are expression containing terms with nonzero coefficient and exponents. For example: $P(x) = a_0 X^n + a_1 X^{n-1} + \dots + a_{n-1} X + a_n$
2. Represent very large numbers and operations of the large number such as addition, multiplication and division.
3. Linked lists are to implement stack, queue, trees and graphs.
4. Implement the symbol table in compiler construction.

Single Linked List:

A linked list allocates space for each element separately in its own block of memory called a "node". The list gets an overall structure by using pointers to connect all its nodes together like the links in a chain. Each node contains two fields; a "data" field to store whatever element, and a "next" field which is a pointer used to link to the next node. Each node is allocated in the heap using malloc(), so the node memory continues to exist until it is explicitly de-allocated using free(). The front of the list is a pointer to the "start" node.



A single linked list

The beginning of the linked list is stored in a "start" pointer which points to the first node. The first node contains a pointer to the second node. The second node contains a pointer to the third node, ... and so on. The last node in the list has its next field set to NULL to mark the end of the list. Code can access any node in the list by starting at the start and following the next pointers.

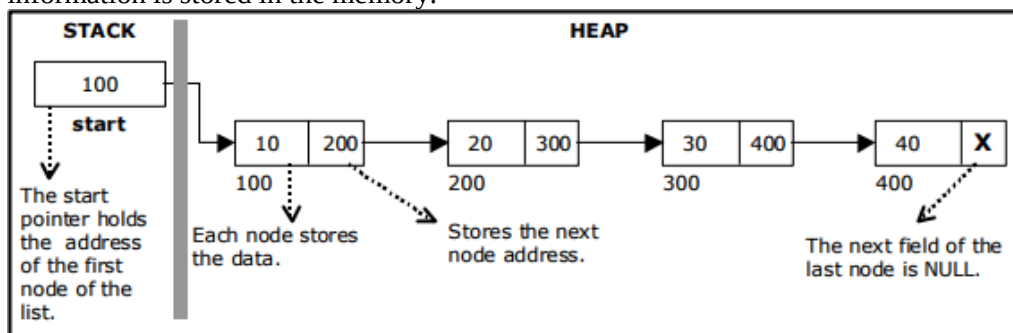
The start pointer is an ordinary local pointer variable, so it is drawn separately on the left top to show that it is in the stack. The list nodes are drawn on the right to show that they are allocated in the heap.

The basic operations in a single linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing.

Creating a node for Single Linked List:

Creating a singly linked list starts with creating a node. Sufficient memory has to be allocated for creating a node. The information is stored in the memory.



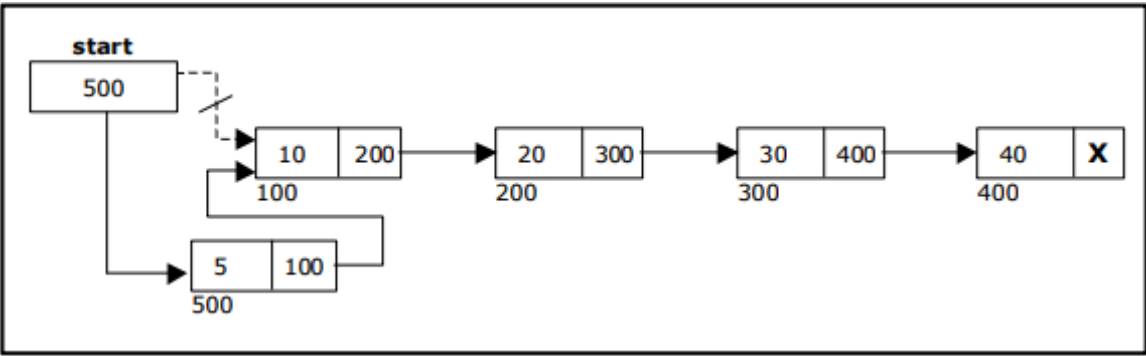
Insertion of a Node:

One of the most primitive operations that can be done in a singly linked list is the insertion of a node. Memory is to be allocated

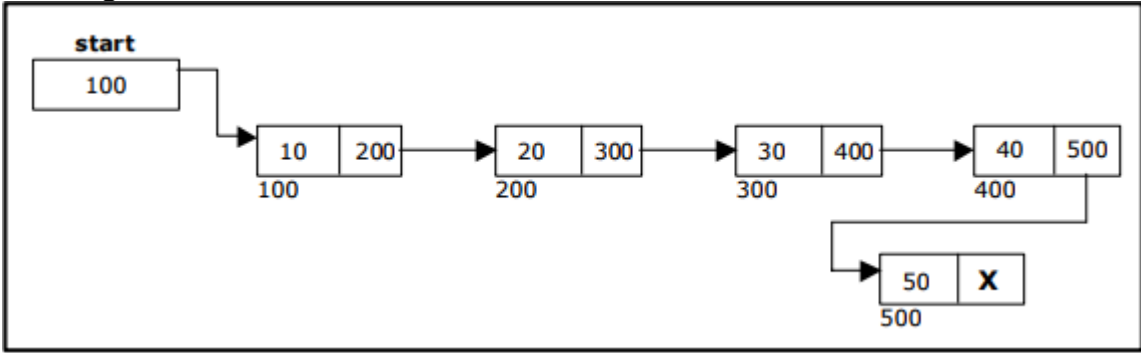
for the new node (in a similar way that is done while creating a list) before reading the data. The new node will contain empty data field and empty next field. The data field of the new node is then stored with the information read from the user. The next field of the new node is assigned to NULL. The new node can then be inserted at three different places namely:

- Inserting a node at the beginning.
- Inserting a node at the end.
- Inserting a node at intermediate position.

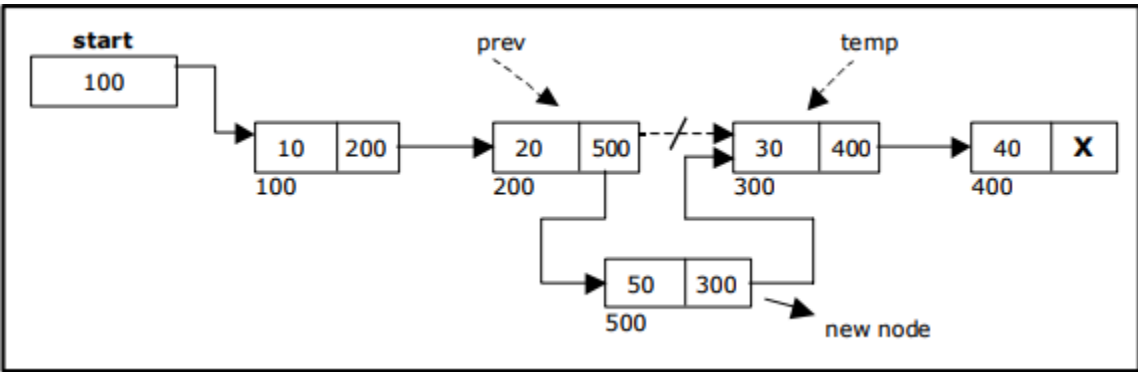
• **Inserting a node at the beginning.**



• **Inserting a node at the end:**



• **Inserting a node into the single linked list at a specified intermediate position other than beginning and end.**

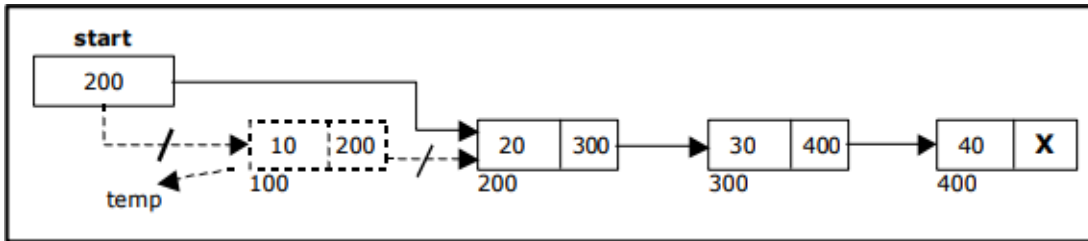


Deletion of a node:

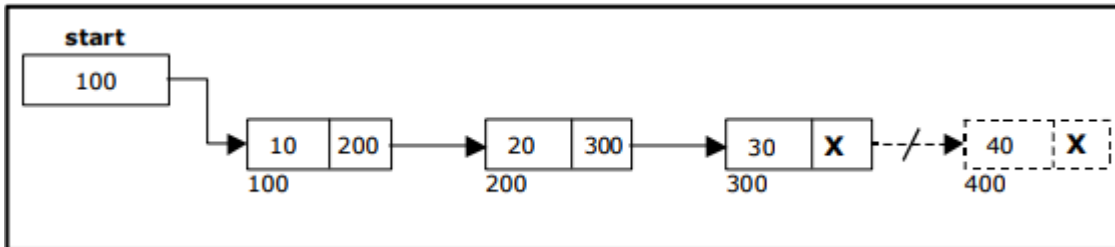
Another primitive operation that can be done in a singly linked list is the deletion of a node. Memory is to be released for the node to be deleted. A node can be deleted from the list from three different places namely.

- Deleting a node at the beginning.
- Deleting a node at the end.
- Deleting a node at intermediate position.

Deleting a node at the beginning:

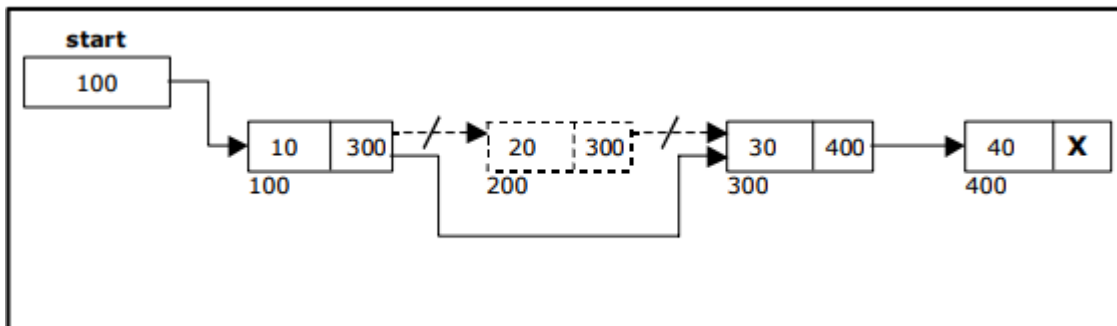


Deleting a node at the end:



Deleting a node at Intermediate position:

The following steps are followed, to delete a node from an intermediate position in the list (List must contain more than two node).



Traversal and displaying a list (Left to Right):

To display the information, you have to traverse (move) a linked list, node by node from the first node, until the end of the list is reached.

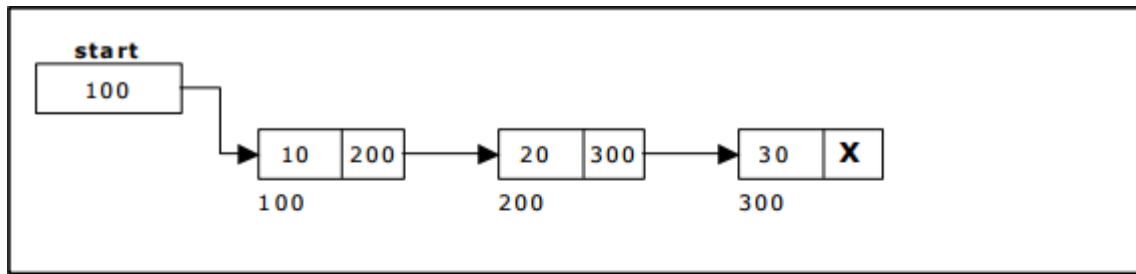
Traversing a list involves the following steps:

- Assign the address of start pointer to a temp pointer.
 - Display the information from the data field of each node. The function traverse () is used for traversing and displaying the information stored in the list from left to right.

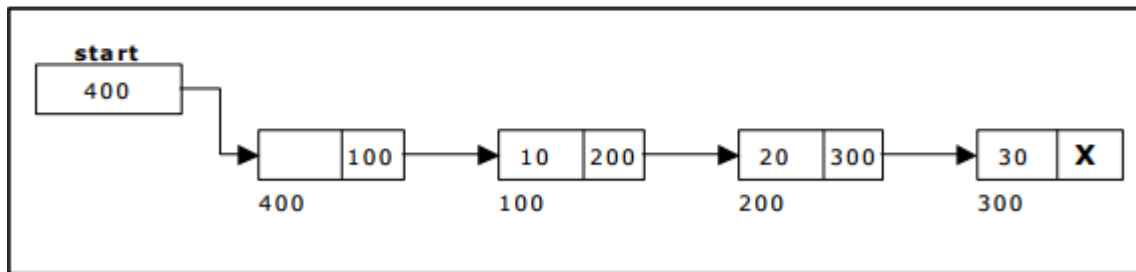
Using a header node:

A header node is a special dummy node found at the front of the list. The use of header node is an alternative to remove the first node in a list. For example, the picture below shows how the list with data 10, 20 and 30 would be represented using a linked list

without and with a header node:



Single Linked List without a header node



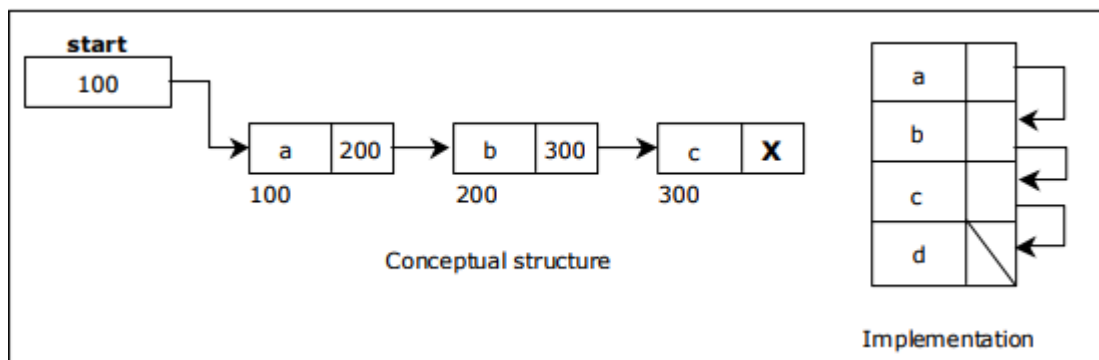
Single Linked List with header node

Note that if your linked lists do include a header node, there is no need for the special case code given above for the remove operation; node n can never be the first node in the list, so there is no need to check for that case. Similarly, having a header node can simplify the code that adds a node before a given node n.

Note that if you do decide to use a header node, you must remember to initialize an empty list to contain one (dummy) node, you must remember not to include the header node in the count of "real" nodes in the list.

It is also useful when information other than that found in each node of the list is needed. For example, imagine an application in which the number of items in a list is often calculated. In a standard linked list, the list function to count the number of nodes has to traverse the entire list every time. However, if the current length is maintained in a header node, that information can be obtained very quickly. 3.5. Array based linked lists: Another alternative is to allocate the nodes in blocks. In fact, if you know the maximum size of a list a head of time, you can pre-allocate the nodes in a single array. The result is a hybrid structure – an array based linked list.

shows an example of null terminated single linked list where all the nodes are allocated contiguously in an array.



Double Linked List: A double linked list is a two-way list in which all nodes will have two links. This helps in accessing both successor node and predecessor node from the given node position. It provides bi- directional traversing. Each node contains three fields:

- Left link.
- Data.
- Right link.

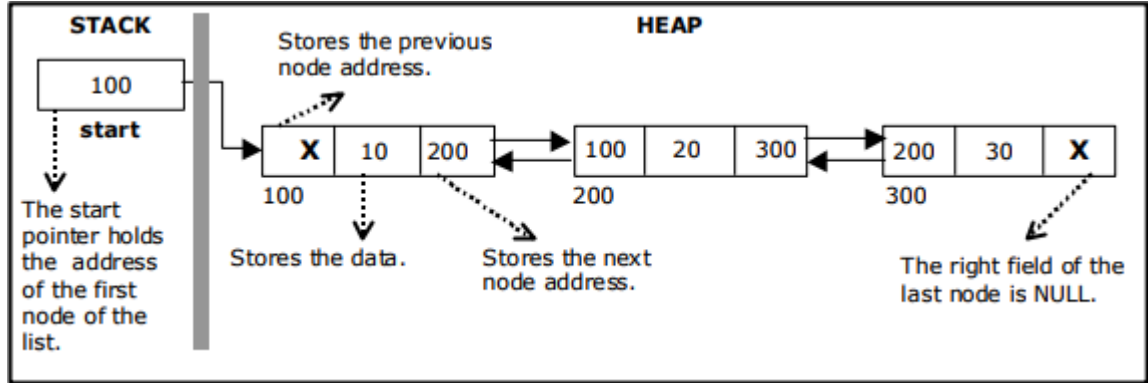
The left link points to the predecessor node and the right link points to the successor node. The data field stores the required data.

Many applications require searching forward and backward thru nodes of a list. For example searching for a name in a telephone directory would need forward and backward scanning thru a region of the whole list.

The basic operations in a double linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing.

A double linked list is shown in figure

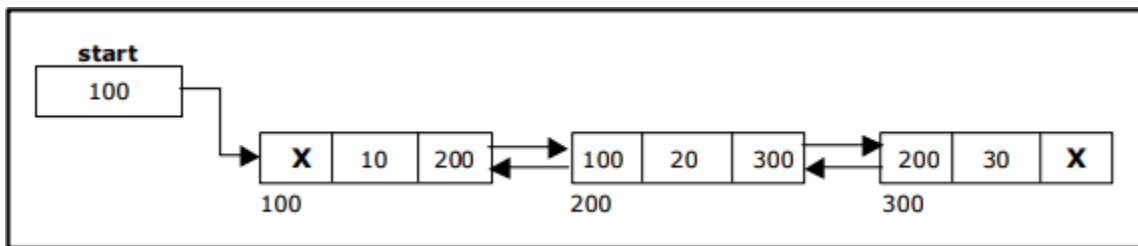


The beginning of the double linked list is stored in a "start" pointer which points to the first node. The first node's left link and last node's right link is set to NULL.

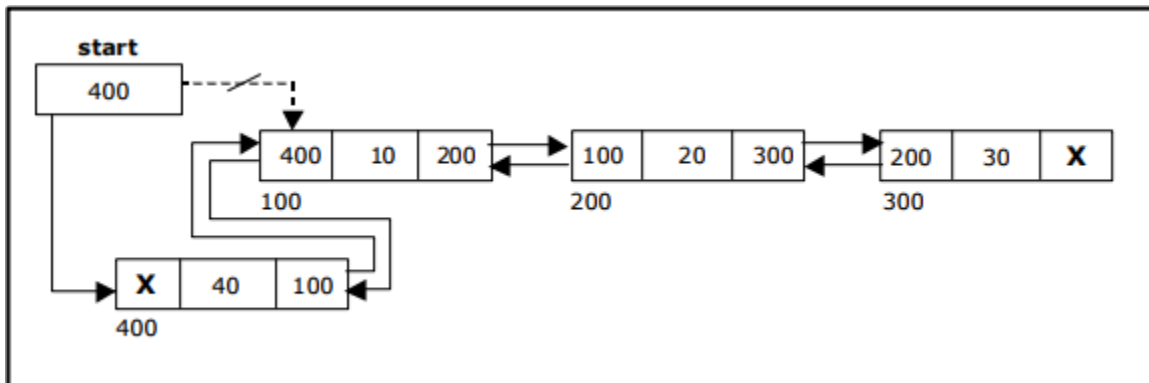
Creating a node for Double Linked List:

Creating a double linked list starts with creating a node. Sufficient memory has to be allocated for creating a node. The information is stored in the memory.

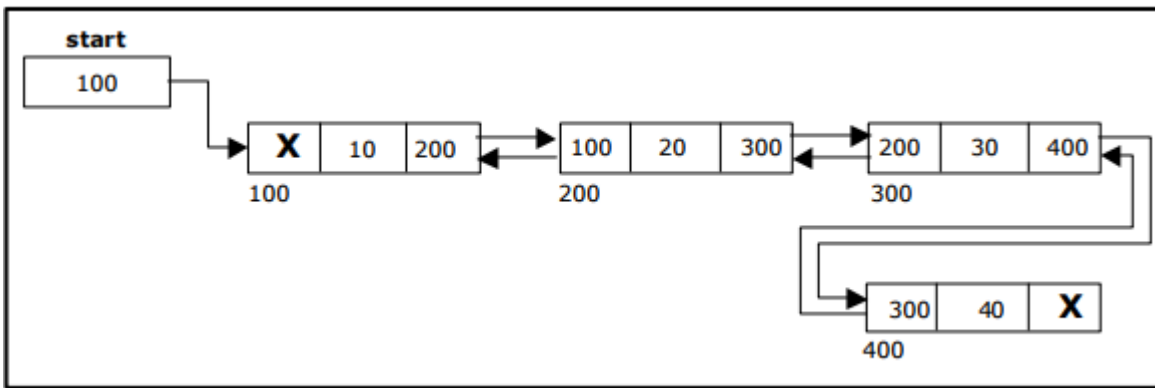
Double Linked List with 3 nodes:



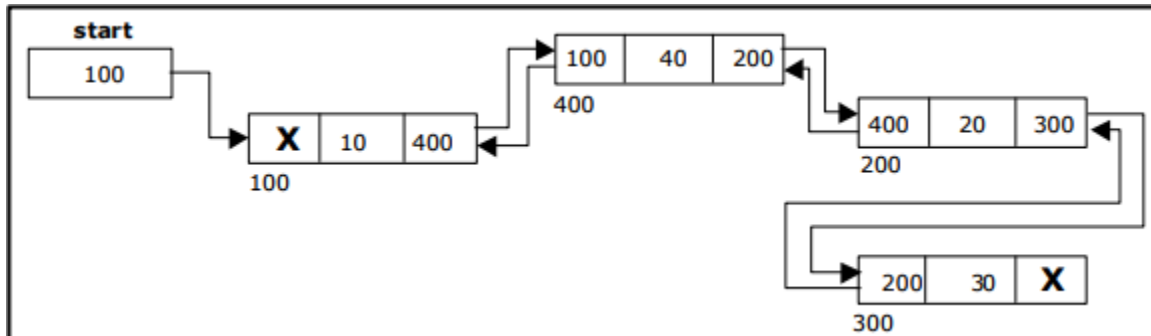
Inserting a node at the beginning:



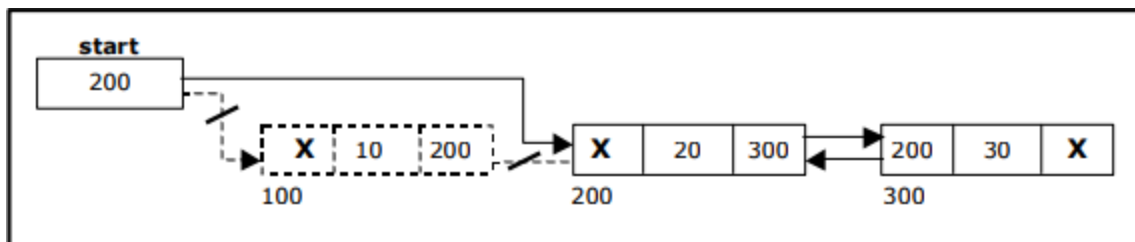
Inserting a node at the end:



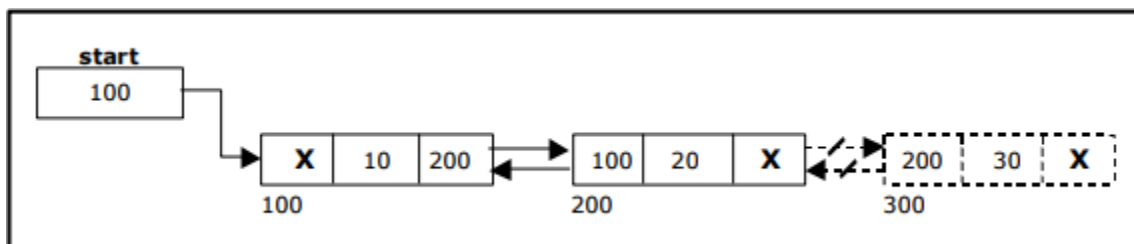
Inserting a node at an intermediate position:



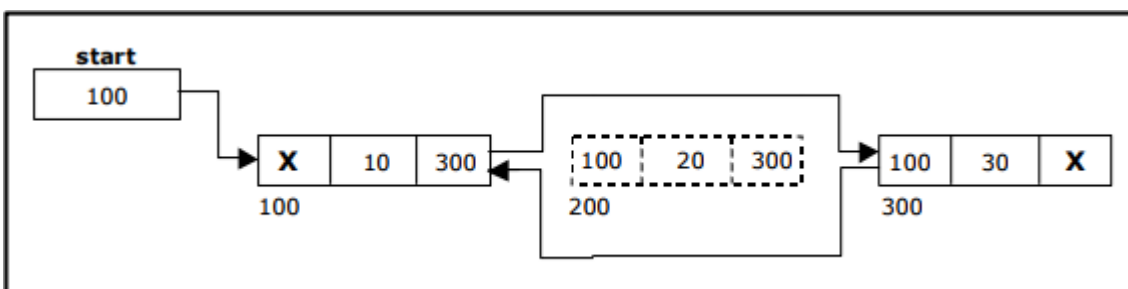
Deleting a node at the beginning:



Deleting a node at the end:



Deleting a node at Intermediate position:



Traversal and displaying a list (Left to Right):

To display the information, you have to traverse the list, node by node from the first node, until the end of the list is reached. The function `traverse_left_right()` is used for traversing and displaying the information stored in the list from left to right.

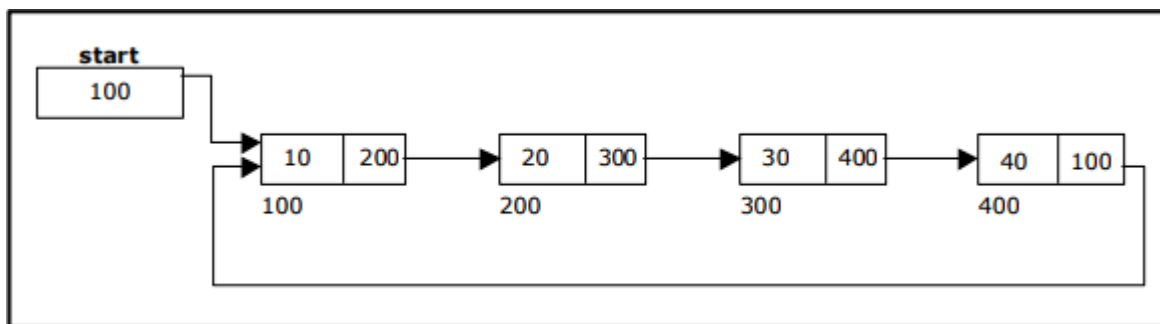
Traversal and displaying a list (Right to Left):

To display the information from right to left, you have to traverse the list, node by node from the first node, until the end of the list is reached. The function `traverse_right_left()` is used for traversing and displaying the information stored in the list from right to left.

Circular Single Linked List:

It is just a single linked list in which the link field of the last node points back to the address of the first node. A circular linked list has no beginning and no end. It is necessary to establish a special pointer called start pointer always pointing to the first node of the list. Circular linked lists are frequently used instead of ordinary linked list because many operations are much easier to implement. In circular linked list no null pointers are used, hence all pointers contain valid address.

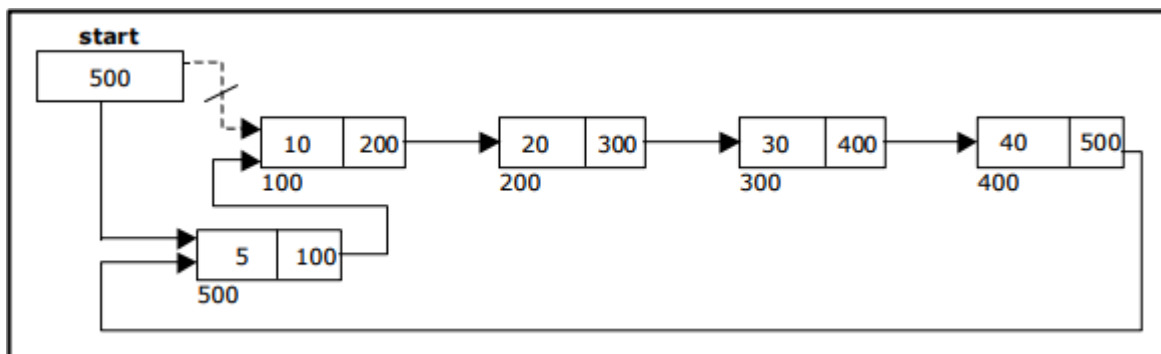
Creating a circular single Linked List with „n“ number of nodes:



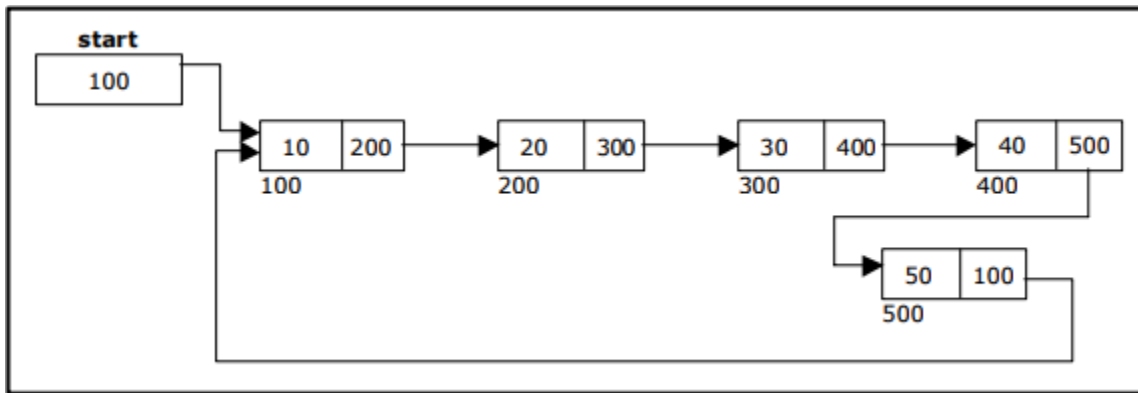
The basic operations in a circular single linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing.

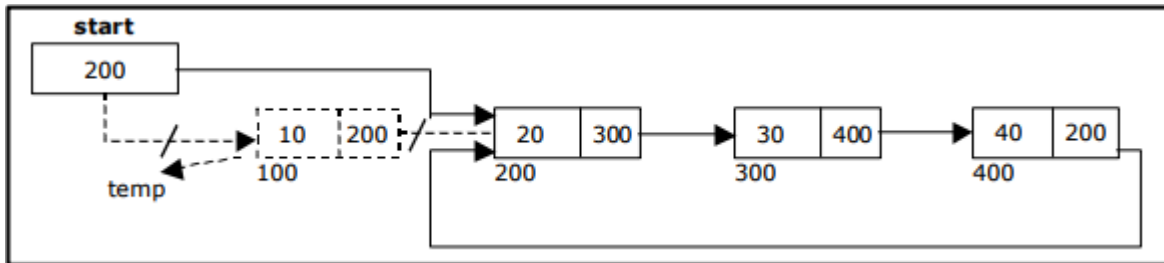
Inserting a node at the beginning:



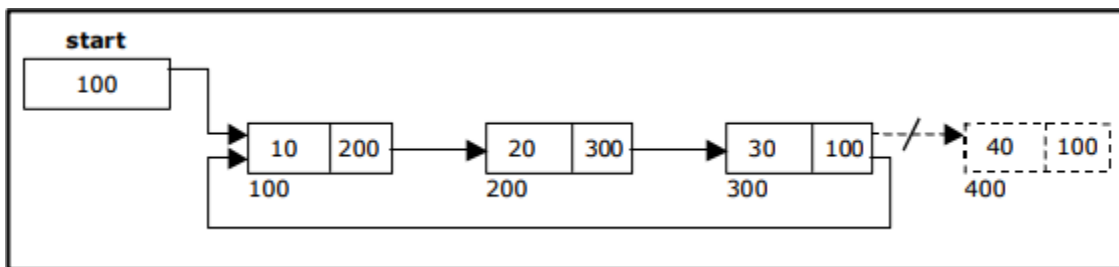
Inserting a node at the end:



Deleting a node at the beginning:



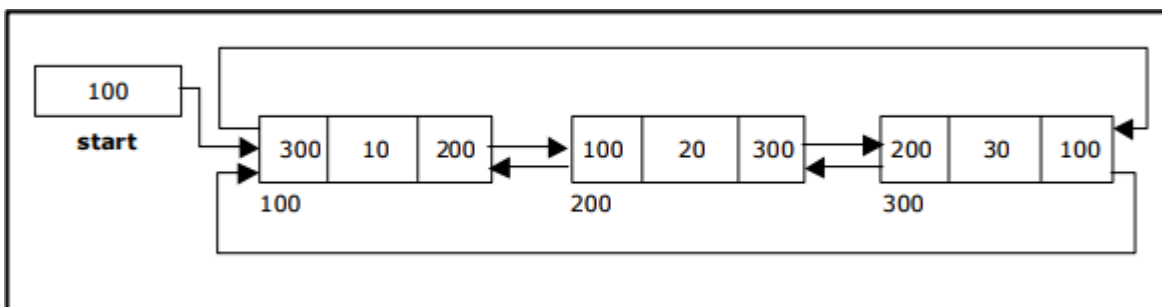
Deleting a node at the end:



Circular Double Linked List:

A circular double linked list has both successor pointer and predecessor pointer in circular manner. The objective behind considering circular double linked list is to simplify the insertion and deletion operations performed on double linked list. In circular double linked list the right link of the right most node points back to the start node and left link of the first node points to the last node.

A circular double linked list is shown in figure

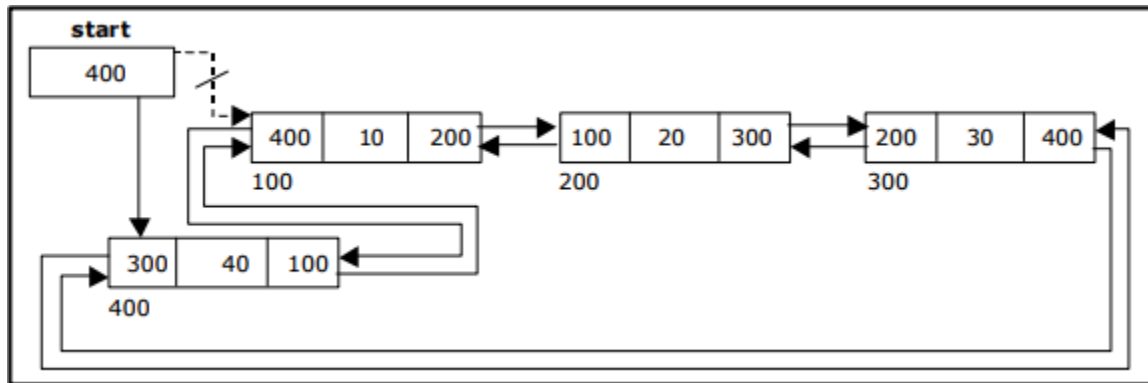


Creating a Circular Double Linked List with „n“ number of nodes

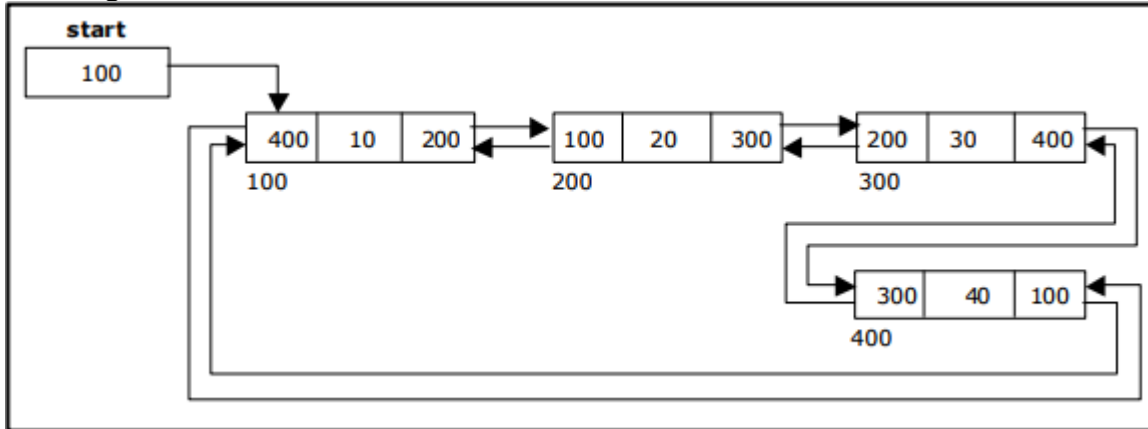
The basic operations in a circular double linked list are:

- Creation.
- Insertion.
- Deletion.
- Traversing.

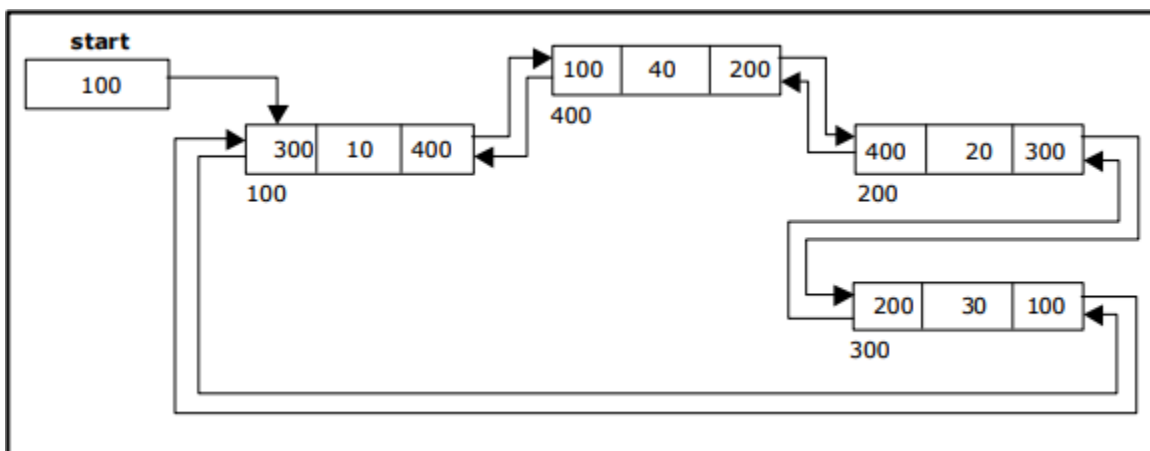
Inserting a node at the beginning:



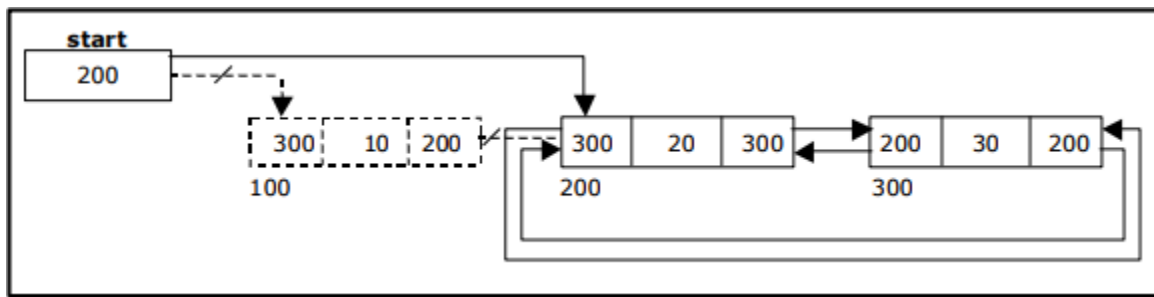
Inserting a node at the end:



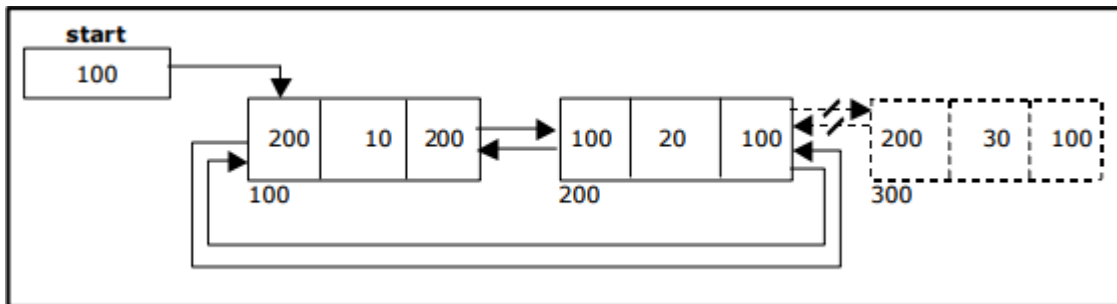
Inserting a node at an intermediate position:



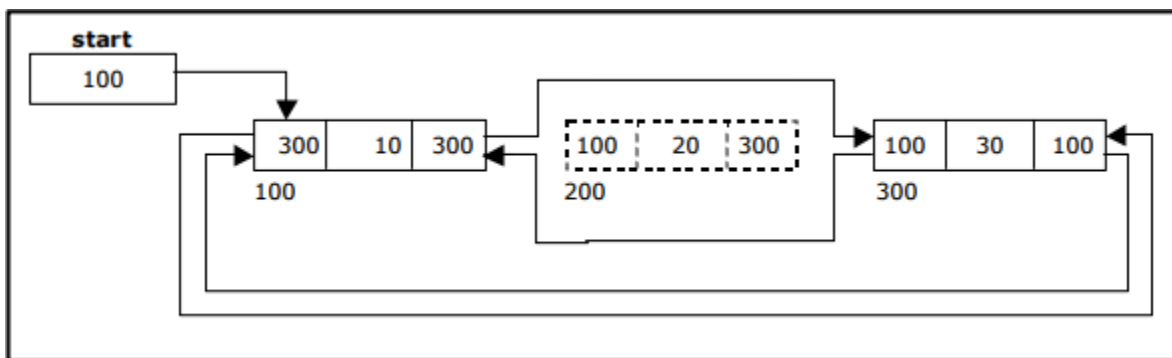
Deleting a node at the beginning:



Deleting a node at the end:



Deleting a node at Intermediate position:



Comparison of Linked List Variations:

The major disadvantage of doubly linked lists (over singly linked lists) is that they require more space (every node has two pointer fields instead of one). Also, the code to manipulate doubly linked lists needs to maintain the prev fields as well as the next fields; the more fields that have to be maintained, the more chance there is for errors.

The major advantage of doubly linked lists is that they make some operations (like the removal of a given node, or a right-to-left traversal of the list) more efficient.

The major advantage of circular lists (over non-circular lists) is that they eliminate some extra-case code for some operations (like deleting last node). Also, some applications lead naturally to circular list representations. For example, a computer network might best be modeled using a circular list.

Polynomials:

A polynomial is of the form: $\sum_{i=0}^n c_i x^i$ Where, c_i is the coefficient of the i th term and

n is the degree of the polynomial Some examples are:

$$5x^2 + 3x + 1$$

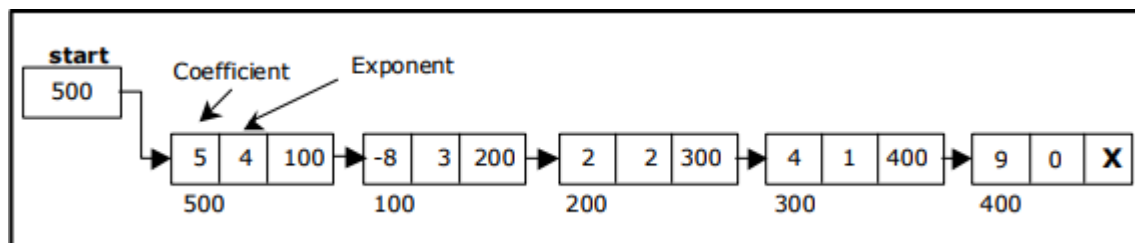
$$12x^3 - 4x$$

$$5x^4 - 8x^3 + 2x^2 + 4x + 9x^0$$

It is not necessary to write terms of the polynomials in decreasing order of degree. In other words the two polynomials $1 + x$ and $x + 1$ are equivalent.

The computer implementation requires implementing polynomials as a list of pairs of coefficient and exponent. Each of these pairs will constitute a structure, so a polynomial will be represented as a list of structures.

A linked list structure that represents polynomials $5x^4 - 8x^3 + 2x^2 + 4x^1 + 9x^0$



Addition of Polynomials:

To add two polynomials we need to scan them once. If we find terms with the same exponent in the two polynomials, then we add the coefficients; otherwise, we copy the term of larger exponent into the sum and go on. When we reach at the end of one of the polynomial, then remaining part of the other is copied into the sum.

To add two polynomials follow the following steps:

- Read two polynomials
- Add them.
- Display the resultant polynomial.